

ECHOBANK PROJECT EXPLAINER

Generated: 2026-08-02

Repository: echobank

1. WHAT THIS PROJECT IS

EchoBank is a prototype banking app designed for people who are fully blind or have very severe visual impairment. The main idea is that banking should not depend on reading a screen. Instead, the app should work through speech, sound, simple touch interactions, and very clear accessible layouts.

This is not just a normal banking app with accessibility added later. The product idea is "voice first". That means the spoken experience is meant to be the primary experience, and the visual screen supports it.

The app is built around a challenge brief about helping a blind banking customer use digital banking independently and safely.

2. THE PROBLEM IT IS TRYING TO SOLVE

Standard banking apps usually assume the user can:

- read balances on screen
- visually scan transactions
- inspect account details before paying someone
- trust a popup confirmation they cannot independently verify
- read letters and statements visually

For a blind customer, that creates dependence on other people and increases the risk of mistakes and scams.

EchoBank tries to solve that by making the key flows work through:

- spoken summaries
- voice interactions
- audio cues
- haptic confirmation
- high-contrast screens as a backup, not the only interface

The goal is independent, safe banking.

3. THE CORE IDEA IN ONE SENTENCE

EchoBank is a voice-first accessible banking app for blind users that helps them check balances, review activity, send money safely, and hear their bank messages without relying on sight.

4. WHAT THE USER CAN DO IN THE APP

Based on the codebase, the app currently includes these areas:

Home / account overview

- hear the available balance
- hear a spoken account summary
- review latest activity
- jump to payments or letters

Payments

- choose an existing payee or add a new one
- enter an amount
- run a scam/risk check
- hear the payment details read back
- confirm a payment with a safer interaction

Activity

- review transactions
- hear a spoken monthly spending summary
- play sound cues for categories

Assistant

- provides example voice commands
- acts like the start of a conversational banking interface
- currently looks more like a limited shell than a fully implemented assistant

Letters / post

- lists bank letters and messages
- reads them aloud
- marks them as read

Settings

- change speaking speed
- toggle automatic speech
- toggle earcons and vibration
- change theme and text size
- freeze or unfreeze card and payments

Pitch page

- there is also a judge-facing presentation route for the competition

5. THE PRODUCT STORY IN SIMPLE LANGUAGE

Imagine a blind customer opening a banking app. Instead of being faced with a screen they must explore visually, the app can immediately say:

"You have 1,284 pounds and 50 pence available. Your latest transaction was 32 pounds and 10 pence at Tesco. You have 2 unread letters."

If they want to send money, the app should help them do it safely by:

- identifying who they are paying
- speaking the amount back clearly
- checking for scam signals
- requiring a deliberate confirmation action

If they receive a letter or statement, they should be able to hear it read aloud rather than needing sighted assistance.

That is the main value proposition: accessible, independent, safer banking.

6. WHAT HAS ACTUALLY BEEN BUILT

The repository already contains a substantial amount of work. This is not just a concept. The code includes:

- a React frontend application
- route-based pages for the main user flows
- a speech and audio interaction layer
- payment and overview flows
- settings and account controls
- backend server functions
- Supabase integration for auth and data
- demo banking data

Important clarification:

The repository history shows that your teammate did not personally build all of the feature work. The history is mostly:

- 2 commits from "Lovable/lovable"
- 64 commits from "gpt-engineer-app[bot]"

So the project exists, but much of the implementation came from bot commits rather than directly from her.

7. HOW THE APP IS STRUCTURED

At a high level, the architecture is:

Frontend

- React app with page routes
- components for UI and accessibility
- voice context for speaking/listening behavior

Interaction layer

- speech synthesis
- some speech recognition support
- audio earcons
- vibration/haptic feedback

Application logic

- account overview
- transactions
- payments
- scam/risk checking
- letters
- settings

Server layer

- typed server functions
- banking queries and mutations
- server-only business logic

Data layer

- Supabase auth
- database tables for accounts, payees, transactions, letters, settings, and related records

AI layer

- AI is intended to help with explanation/summarisation/risk enrichment
- however, some of the current UI suggests parts of the assistant layer are not fully implemented yet

8. THE MAIN ROUTES IN THE APP

These routes exist in the code:

- / landing page
- /auth login/authentication
- /app main authenticated account area
- /app/assistant assistant view
- /app/pay send money flow
- /app/activity transaction history
- /app/letters letters and statements
- /app/settings preferences and controls
- /pitch presentation page

This means the project is already shaped like a working MVP.

9. HOW PAYMENTS WORK

The payment flow is one of the most important parts of the app.

From the code, the flow appears to be:

1. user chooses a saved payee or types a new one
2. user enters an amount
3. the app can save new payee details
4. the app runs a scam check before sending
5. the app speaks back the payment details
6. the user confirms
7. the payment server function executes the transfer

The code also includes a hold-to-confirm component. That matters because the project is trying to avoid unsafe blind confirmation patterns where the user just taps an "OK" button they cannot verify independently.

10. HOW SCAM CHECKING WORKS

There is a deterministic scam/risk engine in the server code.

It looks for things such as:

- new payees
- untrusted payees
- unusually large payments
- out-of-hours activity
- urgency or secrecy language in the request context
- impersonation-style language

This is a strong part of the concept, because accessible banking is not just about readability. It is also about safety and confidence.

11. HOW TRANSACTIONS AND LETTERS WORK

Transactions

- the app can fetch transactions
- render them in a list
- speak individual transaction details
- use category sound cues
- provide a monthly spoken summary

Letters

- the app can fetch letters/messages
- show sender, subject, date, body
- read the content aloud
- mark letters as read

This supports the idea that users should not depend on sight to stay on top of their finances.

12. HOW SETTINGS WORK

The settings route shows that the product is trying to support personal accessibility preferences rather than forcing one mode on every user.

Users can change:

- speaking speed
- whether the app speaks automatically
- sound signatures
- vibration

- theme
- text size

There is also a freeze/unfreeze control for the account, which is a useful safety feature.

13. WHAT THE ASSISTANT LOOKS LIKE RIGHT NOW

The assistant route exists, but the current code suggests it is not yet fully developed as a true conversational banking assistant.

Right now it mainly:

- shows example commands
- responds with a fallback message if it does not handle something

So if someone says "this app has AI", the careful answer is:

"There is an AI direction in the architecture, but the visible assistant experience still looks partial rather than fully mature."

14. WHAT THE DEMO DATA TELLS US

The server code seeds demo data for:

- account details
- payees
- transactions
- letters
- trusted contacts

This means the app is prepared to demo a realistic customer journey without requiring a real bank backend.

That is good for the competition, but it also means you should separate:

- what is demo data
- what is real product behavior

15. WHAT YOUR TEAMMATE APPEARS TO HAVE DONE

Based on git history, the Lovable/lovable commits mainly did:

- initial project scaffold
- repo and tool configuration
- initial README
- later README expansion

The larger feature implementation across routes and logic mostly came from the bot commits in this repository.

So if you are confused about "what she built", that confusion is valid: the repo history is not the same as one teammate manually building the whole product.

16. WHAT IS STRONG ABOUT THE IDEA

The strongest parts of the product idea are:

- clear target user: blind banking customers
- clear problem: normal banking apps are visually biased
- differentiated design: voice first, not screen-reader retrofit
- safety angle: scam/risk checking and explicit confirmation
- accessibility angle: speech, haptics, large controls, settings
- competition fit: easy to explain and socially meaningful

17. WHAT IS WEAK OR UNCLEAR RIGHT NOW

From the codebase, the main weaknesses are:

- the assistant experience looks incomplete
- some parts feel more like an MVP/demo than a finished product
- the architecture is present, but not documented in a very simple way
- the repo history makes ownership confusing
- demo logic and product logic are close together
- the system probably needs stronger testing around core flows

18. WHAT STILL NEEDS TO BE DONE

If the goal is a stronger competition submission or a cleaner product, the likely next steps are:

- make the product story very simple and consistent
- define the user journey clearly
- tighten the payment state flow
- clarify what is deterministic versus AI-driven
- improve or finish the assistant
- add stronger tests for critical flows
- separate demo setup from core business logic
- prepare a clear architecture explanation for judges/team members

19. THE SIMPLEST WAY TO EXPLAIN THE PROJECT TO SOMEONE

If you need to explain it in plain language, say this:

"We are building a banking app for blind users. Instead of depending on reading a screen, it uses speech, sound cues, and simple accessible interactions so a blind customer can check their balance, review transactions, send money safely, and hear their bank messages."

20. FINAL SUMMARY

EchoBank is a voice-first accessible banking MVP for blind users. It is meant to help someone bank independently and safely without relying on vision. The app already includes the main screens and a lot of working demo logic, especially for home overview, payments, activity, letters, and settings. The biggest remaining challenge is not inventing the idea from scratch. The challenge is understanding the current MVP, cleaning up the architecture story, and making the core flows feel complete, credible, and ready to present.